

printing the maze itself. Without this HTML tag, the maze will appear distorted in the browser.

Within the function, nested `for` loops loop over every space in the maze data structure. These `for` loops iterate over each y-coordinate from `0` up to, but not including, `HEIGHT`, and each x-coordinate from `0` up to, but not including, `WIDTH`.

Inside the inner `for` loop, if the current x, y coordinates match the position of the mark (the location where the algorithm is currently carving), the program displays the `@` in the `MARK` constant. The Python code does this as follows:

Python

```
        if markX == x and markY == y:
            # Display the '@' mark here:
            print(MARK, end='')
        else:
            # Display the wall or empty space:
            print(maze[(x, y)], end='')

    print() # Print a newline after printing the row.
```

The JavaScript code is as follows:

JavaScript

```
        if (markX === x && markY === y) {
            // Display the '@' mark here:
            document.write(MARK);
        } else {
            // Display the wall or empty space:
            document.write(maze[(x, y)]);
        }
    }
    document.write('<br />'); // Print a newline after printing the row.
}
document.write('</code>');
}
```
